



LEKTION 18

OBSERVER

MÅL

EFTER ENDT UNDERVISNING KAN DEN STUDERENDE:

- Redegøre for observer
- Udfærdige et eksempel og anvendelse af observer

RECAP

1. Recap
2. Beskrivelse
3. Behov
4. Løsning
5. Opgave

BESKRIVELSE

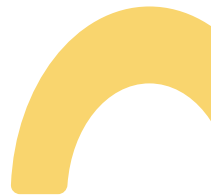
EN GIVEN WEBSHOP SKAL UDVIDE DERES FUNKTIONALITET, SÅLEDES NÅR EN ORDRE ÆNDRES SKAL FØLGENDE HANDLINGER SKE:

- Opdatere UI
- Logfør ændring
- Skriv mail ud til kunden
- Opdatere lager status.

Dette sikre at kunden for rettidig besked om eventuel leverance, men at der også er et intern spor på hvad der er sket hvornår.



BEHOV





BEHOV

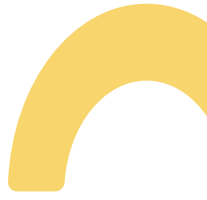
PROBLEM

Dette er yderst kompleks, ufleksibelt og skaber dybe afhængigheder. Så der ønskes at finde en bedre løsning.



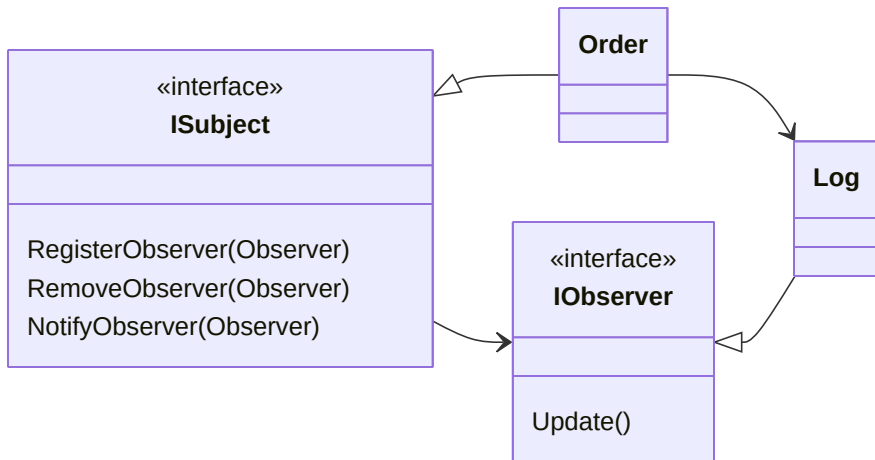
BEHOV

Vi ønsker:

- Ordre skal kun sige: "Noget er ændret"
 - Andre dele kan lytte
 - Let at tilføje nye reaktioner
- 
- 

LØSNING

DIAGRAM



KODE

```
IObserver[] observers = new IObserver[]
{
    new Log(),
    new Ui(),
    new Mail(),
    new Stock()
};

ISubject order = new Order();

Console.WriteLine("Register Observers");
foreach (IObserver observer in observers)
{
    order.RegisterObserver(observer);
}

Console.WriteLine("Order change");
order.ChangeState("With currier");
```

OPGAVE 1 - IDENTIFICER OBSERVERS (LET)

Webshoppen skal reagere på ændringer i en ordre.

Når en ordre ændres skal følgende ske:

- UI opdateres
- ændringen logføres
- kunden modtager en mail
- lageret opdateres

Opgave

1. Identificer Subject
2. Identificer Observers

Tegn et simpelt klassediagram med:

1. ISubject
2. IObserver
3. Order
4. mindst to observers

OPGAVE 2 - IMPLEMENTER OBSERVER (GRUNDNIVEAU)

Implementer følgende interfaces:

```
interface IObserver
{
    void Update();
}

interface ISubject
{
    void RegisterObserver(IObserver observer);
    void RemoveObserver(IObserver observer);
    void NotifyObservers();
}
```

OPGAVE

1. Lav en klasse Order der implementerer ISubject
2. Lav en observer Log
3. Registrer Log som observer
4. Når Order.ChangeStatus() kaldes skal Log.Update() kaldes

TEST

```
Order order = new Order();
order.RegisterObserver(new Log());

order.ChangeStatus();
```

OPGAVE 3 - FLERE OBSERVERS (MELLEMNIVEAU)

Udvid systemet så flere observers kan reagere på ændringer.

Tilføj følgende observers:

1. Ui
2. Mail
3. Stock

Når en ordre ændres skal alle observers opdateres.

Opgave

1. Implementer de tre observers
2. Registrer dem i Order
3. Kald NotifyObservers() når status ændres

Output kunne fx være:

```
Log: Order changed
Ui: Updating layout
Mail: Sending mail to customer
Stock: Updating stock
```

OPGAVE 4 - UDVID SYSTEMET (SVÆR)

WEBSHOPPEN ØNSKER NU AT KUNNE TILFØJE NYE REAKTIONER UDEN AT ÆNDRE ORDER.

Tilføj en ny observer:

```
SmsNotification
```

Denne skal sende en SMS når en ordre ændres.

Opgave

1. Implementer SmsNotification
2. Registrer den som observer
3. Test at alle observers reagerer

Refleksion:

Hvad skulle ændres hvis systemet ikke brugte Observer pattern?

Hvorfor gør Observer systemet mere fleksibelt?

